# Binary Search Pattern -- Complete Guide

# Binary Search Pattern

## 1. What is it, and when is it used?

**Binary search** is a divide-and-conquer search technique that repeatedly halves a **monotonic search space** to find a target value or the boundary where a condition flips from false to true (or true to false). Classic form: search a sorted array in O(log n) by comparing the middle element and discarding half the space each time.

It is used any time the search space is **sorted, or can be framed as monotonic** (there's a clean "cutoff point" -- everything before it fails a condition, everything after it passes, or vice versa), even if there's no explicit array to search -- the space can be a range of *answers* (e.g., "minimum speed", "maximum capacity").

## 2. Why is it used?

Linear scan of a sorted space is O(n); binary search collapses this to O(log n), which is often the difference between passing and timing out on large inputs (n up to 1e9 or answer ranges up to 1e18). It also elegantly solves **"find the smallest/largest value satisfying condition X"** problems where brute force would mean trying every candidate value one by one -- binary search on the *answer* tests O(log(range)) candidates instead of all of them.

## 3. Where is it used?

- Searching in a sorted array (classic lookup, first/last occurrence)
- Finding insertion point ( `lowerBound` / `upperBound` , `bisect` )
- Searching in a rotated sorted array
- **Binary search on the answer**: minimize max/maximize min type problems (e.g., "Koko eating bananas", "capacity to ship packages in D days", "split array largest sum", "minimum days to make m bouquets")
- Searching in 2D sorted matrices
- Finding peak elements in an array (bitonic search)
- Searching in infinite/unbounded sorted structures (exponential + binary search)
- Median of two sorted arrays
- Square root / kth root computation, floating-point precision search

## 4. How do you identify it's a Binary Search problem? (Pattern Triggers)

Ask yourself: **"Is there a sorted/monotonic property -- a point where the answer flips from 'no' to 'yes' -- that I can exploit instead of checking every candidate?"** If yes -> binary search.

**Signal phrases (the 2-minute test):** - "sorted array" / "rotated sorted array" appears in the problem - "find the minimum/maximum value such that..." (classic binary-search-on-answer) - "find the Kth smallest/largest" combined with a range of possible values - words like "at least", "at most", "minimum possible", "maximum possible" paired with a feasibility check - O(log n) is hinted by constraints (n up to 1e5-1e9, but expected complexity is log) - "first/last position of element", "insertion point", "closest element" - a function is described as monotonic (increasing/decreasing) even if not stated as "sorted"

## 5. Can you identify it from constraints?

| Constraint clue | Implication |
|---|---|
| Array explicitly sorted, n up to 1e6-1e9 | Classic binary search, O(log n) |
| n up to 1e5 but answer range is huge (1e9, 1e18) | Binary search on the answer, not on the array |
| "Minimize the maximum" / "maximize the minimum" phrasing | Binary search on answer + greedy feasibility check |

| | |
|---|---|
| Need O(log n) or O(n log n) overall and array is sorted | Binary search is the intended tool |
| Data is a monotonic function (feasibility check is monotonic even without a data structure) | Binary search on answer space |
| Array not sorted and no monotonic structure exists | Binary search does NOT apply -- reconsider |

## 6. Types / Variants

1. **Classic search** -- find exact target in sorted array, return index or -1.
2. **Lower bound / Upper bound** -- first index >= target, first index > target (a.k.a. `bisect_left` / `bisect_right`).
3. **First/Last occurrence** -- find boundaries of a range of equal elements.
4. **Binary search on rotated sorted array** -- determine which half is sorted, then decide direction.
5. **Binary search on the answer** -- search over a range of possible answers using a monotonic feasibility/predicate function ( `canAchieve(mid)` ), not over an array.
6. **Binary search on 2D matrix** -- treat as a flattened 1D sorted array or search row/column-wise.
7. **Peak finding (bitonic search)** -- array increases then decreases; find the peak using comparison with neighbors.
8. **Exponential/unbounded binary search** -- for infinite/unknown-length sorted structures, first find a bound by doubling, then binary search within it.
9. **Ternary search** -- variant for unimodal (single peak/valley) continuous functions.

## 7. Rationale / Intuition

Binary search works because a monotonic property lets you **eliminate half the remaining candidates with a single comparison** -- you never need to re-examine the discarded half because the property guarantees it can't contain the answer. Think of it as playing "20 questions" optimally: each yes/no query ( `is mid too small/large?` or `does mid satisfy the condition?` ) halves the uncertainty, so you need only `log2(n)` queries to pinpoint the answer among n candidates.

The generalization to "binary search on the answer" is the deeper insight: you don't need a literal sorted array -- you need a **predicate** `feasible(x)` that is monotonic (false...false, true...true, with one flip point). Once you can write that predicate and check it in reasonable time, binary search finds the flip point regardless of what "x" represents (a speed, a capacity, a distance, a day count).

## 8. Generic Time & Space Complexity

- **Time:** O(log n) for classic search over n elements; O(log(range) x cost of feasibility check) for binary-search-on-answer (often O(n log(range)) total if the check itself is O(n)).
- **Space:** O(1) iterative; O(log n) if implemented recursively (call stack).

## 9. Comparison With Other Search Patterns

| Pattern | Search space | Precondition | Time | Best when |
|---|---|---|---|---|
| **Linear search** | Any array | None | O(n) | Unsorted, small n |
| **Binary search (classic)** | Sorted array | Array sorted | O(log n) | Direct lookup in sorted data |
| **Binary search on answer** | Range of possible answers | Feasibility predicate is monotonic | O(log(range) x check) | "Min/max value satisfying condition" problems |
| **Two pointers** | Sorted array / string | Sorted or specific structure | O(n) | Pair-sum, merging, in-place partitioning |
| **Hashing** | Any array | None | O(n) avg | Existence/frequency lookups, no order needed |
| **Ternary search** | Unimodal function | Single peak/valley | O(log n) | Continuous |

| | | | | |
|---|---|---|---|---|
| **Ternary search** | Unimodal function | Single peak/valley | O(log n) | optimization, not exact match |

**Rule of thumb:** if data is sorted or the answer space is monotonic, prefer binary search over linear scan or brute force -- it's almost always safe to reach for it first when you see "sorted" or "minimize the maximum."

## 10. Reusable Java Template

```
public class BinarySearch {

    // 1. Classic exact search
    public static int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;
            else if (nums[mid] < target) lo = mid + 1;
            else hi = mid - 1;
        }
        return -1; // not found
    }

    // 2. Lower bound: first index with nums[i] >= target
    public static int lowerBound(int[] nums, int target) {
        int lo = 0, hi = nums.length; // note: hi = length, exclusive
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] < target) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }

    // 3. Upper bound: first index with nums[i] > target
    public static int upperBound(int[] nums, int target) {
        int lo = 0, hi = nums.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] <= target) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }

    // 4. Binary search on the answer (minimize the max / find smallest feasible value)
    //    Template: find smallest x in [lo, hi] such that feasible(x) is true
    public static int binarySearchOnAnswer(int lo, int hi) {
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (feasible(mid)) {
                hi = mid;       // mid works; try smaller
            } else {
                lo = mid + 1;   // mid doesn't work; go bigger
            }
        }
        return lo; // smallest feasible value
    }

    private static boolean feasible(int x) {
        // Problem-specific monotonic predicate, e.g.:
        // "can we ship all packages within D days if capacity == x?"
        return true; // placeholder
    }
```

```
        // 5. Search in rotated sorted array
        public static int searchRotated(int[] nums, int target) {
            int lo = 0, hi = nums.length - 1;
            while (lo <= hi) {
                int mid = lo + (hi - lo) / 2;
                if (nums[mid] == target) return mid;

                if (nums[lo] <= nums[mid]) { // left half is sorted
                    if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
                    else lo = mid + 1;
                } else { // right half is sorted
                    if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
                    else hi = mid - 1;
                }
            }
            return -1;
        }

        // 6. Find peak element (bitonic search)
        public static int findPeak(int[] nums) {
            int lo = 0, hi = nums.length - 1;
            while (lo < hi) {
                int mid = lo + (hi - lo) / 2;
                if (nums[mid] > nums[mid + 1]) hi = mid;    // peak in left half (incl. mid)
                else lo = mid + 1;                          // peak in right half
            }
            return lo;
        }
    }
```

## 11. Quick Recognition Checklist (2-minute test)

- ☐ Is the array/data explicitly sorted or rotated-sorted?
- ☐ Does the problem ask for "minimum/maximum value such that some condition holds"?
- ☐ Is there a monotonic feasibility check ( `feasible(x)` flips exactly once as x increases)?
- ☐ Are constraints hinting at O(log n) or O(n log n) expected complexity?

If 2+ checked -> Binary Search.